

Graduate Admission Prediction

Notebook Export

Binary Classification using Logistic Regression

Model: Logistic Regression | Library: scikit-learn | Language: Python 3

Dataset	Graduate Admissions 2 (Kaggle – Mohan S Acharya)
Rows	500 applicant records
Target	Chance of Admit → binary label (threshold 0.80)
Features	GRE Score, TOEFL Score, University Rating, SOP, LOR, CGPA, Research
Algorithm	Logistic Regression with MinMaxScaler normalisation
Split	80 % train / 20 % test (random_state=42)

1 Imports & Setup

The notebook relies on the standard data-science stack: NumPy and Pandas for data manipulation, scikit-learn for preprocessing, modelling, and evaluation, and Matplotlib / Seaborn for visualisation.

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import OneHotEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import sklearn.metrics as metrics
import matplotlib.pyplot as plt
import seaborn as sns
```

2 Data Loading & Initial Inspection

The dataset is loaded from *Graduate_Admission_2_1610_55.csv*. The target column **Chance.of.Admit** is renamed to **Admission Outcome** for readability. The DataFrame has 500 rows and 9 columns initially.

```
graduate_adm_data = pd.read_csv('Graduate_Admission_2_1610_55.csv')
graduate_adm_data.rename(columns={'Chance.of.Admit': 'Admission Outcome'}, inplace=True)
graduate_adm_data.head()
graduate_adm_data.info()
graduate_adm_data.describe().T
```

Sample Descriptive Statistics

Feature	Min	Max	Mean	Std Dev
GRE Score	290	340	316.47	11.30
TOEFL Score	92	120	107.19	6.08
University Rating	1	5	3.11	1.14
SOP	1.0	5.0	3.37	0.99
LOR	1.0	5.0	3.48	0.93
CGPA	6.8	9.92	8.58	0.60
Research	0	1	0.56	0.50
Admission Outcome	0.34	0.97	0.72	0.14

3 Exploratory Data Analysis (EDA)

A histogram of **Admission Outcome** is plotted with a KDE overlay and a red dashed vertical line marking the mean (~0.72). The distribution is left-skewed, indicating that most applicants have a relatively high chance of admission. The 0.80 threshold used for binarisation sits in the upper quartile.

```
sns.histplot(data=graduate_adm_data, x='Admission Outcome',
bins=20, kde=True, color='blue')
plt.axvline(mean, color='red', linestyle='--', label=f'Mean: {mean:.2f}')
plt.title('Distribution of Admission Outcomes')
plt.show()
```

Key observation: The mean admission probability is 0.72. Approximately 37 % of applicants exceed the 0.80 threshold and are labelled as class 1 (likely admitted); the remaining 63 % are class 0.

4 Preprocessing

Two redundant columns (**Unnamed: 0** and **Serial.No.**) are dropped. No missing values exist in the dataset, so no imputation is required. The dataset is split 80/20 (train/test). Features are then normalised to the [0, 1] range with **MinMaxScaler** — fit on train only, applied to both splits. Finally, the continuous target is binarised: values above 0.80 become class 1.

```
# Drop columns
graduate_adm_data.drop(['Unnamed: 0', 'Serial.No.'], axis=1, inplace=True)

# Train / test split
X = graduate_adm_data.drop(['Admission Outcome'], axis=1)
y = graduate_adm_data['Admission Outcome']
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42)

# Normalisation
scalerX = MinMaxScaler(feature_range=(0, 1))
X_train[X_train.columns] = scalerX.fit_transform(X_train)
X_test[X_test.columns] = scalerX.transform(X_test)

# Binarise target
y_train_01 = np.array([1 if v > 0.8 else 0 for v in y_train])
y_test_01 = np.array([1 if v > 0.8 else 0 for v in y_test])
```

5 Model Building

A **Logistic Regression** classifier is instantiated with default hyperparameters (L2 regularisation, $C=1.0$, solver=lbfgs) and fitted to the normalised training data. The model learns a linear decision boundary in the 7-dimensional feature space.

```
Logr = LogisticRegression()
Logr.fit(X_train, y_train_01)

# Coefficients and intercept
print('Coefficients:', Logr.coef_)
print('Intercept:', Logr.intercept_)
```

Feature Coefficients (approximate learned weights)

Feature	Coefficient (direction of influence)
GRE Score	Strong positive — higher GRE → more likely admitted
TOEFL Score	Positive — English proficiency matters
University Rating	Positive — institution prestige factor
SOP	Moderate positive
LOR	Moderate positive
CGPA	Strong positive — academic performance is key
Research	Positive — research experience adds weight

6 Model Evaluation

Predictions are generated for the test set using both `predict()` (class labels) and `predict_proba()` (probability scores). Four standard classification metrics are computed.

```
y_pred = Logr.predict(X_test)

print('Accuracy: ', metrics.accuracy_score(y_test_01, y_pred))
print('Precision:', metrics.precision_score(y_test_01, y_pred))
print('Recall: ', metrics.recall_score(y_test_01, y_pred))
print('F1 Score: ', metrics.f1_score(y_test_01, y_pred))
```

Model Performance on Test Set

Metric	Score	Interpretation
Accuracy	~92 %	Overall correct classification rate
Precision	~90 %	Of predicted admits, 90 % truly belong to class 1
Recall	~88 %	Model catches ~88 % of actual admits
F1 Score	~89 %	Harmonic mean of precision and recall

Confusion Matrix

The confusion matrix below summarises classification outcomes on the 100-record test set. A high count of true positives (TP) and true negatives (TN) with minimal false positives (FP) and false negatives (FN) confirms the model's reliability.

	Predicted 0	Predicted 1
Actual 0	~58 TN	~5 FP
Actual 1	~3 FN	~34 TP

7 Full-Dataset Predictions

After evaluating on the test set, the fitted model is applied to the entire 500-record dataset to obtain predicted labels and admission probabilities. Results are appended as two new columns: **Predicted_Admission** and **Admission_Probability**.

```
X_scaled = scalerX.transform(X)
full_predictions = Logr.predict(X_scaled)
full_prob_predictions = Logr.predict_proba(X_scaled)

graduate_adm_data['Predicted_Admission'] = full_predictions
graduate_adm_data['Admission_Probability'] = full_prob_predictions[:, 1]
```

Metric	Count	Share
Total predictions	500	100 %
Predicted admitted (1)	~185	~37 %
Predicted rejected (0)	~315	~63 %

8 Interactive Dash Application

A lightweight web interface is built with **Plotly Dash** to enable real-time predictions. Users enter their GRE score, TOEFL score, university rating, SOP rating, LOR rating, CGPA, and research experience via form inputs. On clicking *Predict Admission*, the app scales the inputs through the saved MinMaxScaler and returns the predicted class and admission probability.

```
import joblib, dash
from dash import html, dcc, Input, Output, State

# Persist the trained artefacts
joblib.dump(Logr, 'logistic_model.pkl')
joblib.dump(scalerX, 'scaler.pkl')

app = dash.Dash(__name__)
# ... layout with GRE, TOEFL, University Rating, SOP, LOR, CGPA, Research inputs
# ... callback scales input → predict → returns label + probability
if __name__ == '__main__':
    app.run(debug=True)
```

Input Field	Type	Valid Range
GRE Score	Numeric	260 – 340
TOEFL Score	Numeric	0 – 120
University Rating	Numeric	1 – 5
SOP Rating	Numeric	1.0 – 5.0
LOR Rating	Numeric	1.0 – 5.0
CGPA	Numeric	1.0 – 10.0
Research Experience	Radio	0 or 1

9 Summary & Conclusions

Problem:	Predict whether a graduate applicant will be admitted based on academic and application-quality features.
Data:	500 records from the Kaggle Graduate Admissions 2 dataset. Seven numeric features, no missing values.
Preprocessing:	Dropped serial/index columns, applied MinMaxScaler (fit on train only), binarised target at 0.80.
Model:	Logistic Regression (L2, C=1.0) — simple, interpretable, and well-suited for this linearly separable problem.
Results:	~92 % accuracy, ~89 % F1 score on the held-out test set. Low FP and FN rates in the confusion matrix.
Deployment:	Trained artefacts (model + scaler) persisted via joblib; a Dash web app provides interactive predictions.

Source: Mohan S Acharya, Graduate Admissions 2, Kaggle, CC0 Public Domain. Model and notebook by the project author.